

Defensa oral — TPO Programación I (Sanatorio San Roque)

1. Idea general del programa

Es un simulador de reserva de turnos para un sanatorio ficticio (Sanatorio San Roque). Por consola, el programa:

1. Pide los datos de un paciente.
2. Le hace elegir especialidad médica y práctica/estudio dentro de esa especialidad.
3. Le muestra una grilla de horarios (días × franjas horarias) y le hace elegir un turno libre.
4. Calcula el costo según si tiene obra social o no.
5. Genera un comprobante con formato prolijo.
6. Repite esto para tantos pacientes como se quiera, en un menú con loop.
7. Al finalizar, muestra un resumen de todos los turnos reservados en la sesión.

El estado del programa vive todo en memoria (nada se guarda en disco): dos listas globales (DIAS_SEMANA, FRANJAS_HORARIAS), la matriz `matriz_turnos` que se va modificando turno a turno, y `lista_turnos` que acumula el historial de reservas de la corrida.

2. Estructura del archivo

Constantes globales (línea 2-3)
`DIAS_SEMANA, FRANJAS_HORARIAS`

Funciones (línea 6-172)
`datos_paciente()`
`seleccionar_estudio(especialidades, cardio, trauma, derma, pediatra)`
`mostrar_disponibilidad(matriz_turnos)`
`elegir_turno(matriz_turnos)`
`obra_social()`
`formatear_monto(monto)`
`generar_comprobante(datos, especialidad, estudio, dia, hora, obra_social_nombre, monto)`
`mostrar_resumen_turnos(lista_turnos)`

Programa principal (línea 175-207)
inicialización de datos (`especialidades, matriz, lista_turnos`)
loop del menú principal

Todas las funciones están definidas antes del programa principal, que es el único bloque con código “suelto” (fuera de funciones). Esto es intencional: cada función resuelve una responsabilidad puntual y el programa principal solo las orquesta.

3. Detalle función por función

`datos_paciente()` MARTI

Qué hace: pide y valida los datos del paciente: nombre y apellido, edad, mail y DNI.

Cómo valida: cada dato usa un `while` que se repite mientras el valor sea inválido — es el patrón “pedir dato → validar → si es inválido, volver a pedir” repetido cuatro veces: - Nombre: longitud entre 5 y 25 caracteres. Se guarda con `.title()` para capitalizar cada palabra (ej. “juan perez” → “Juan Perez”). - Edad: entre 0 y 110. - Mail: longitud entre 10 y 200 caracteres, se guarda en minúsculas con `.lower()`. - DNI: se valida con `.isdigit()` (que sea todo números) y que tenga 7 u 8 dígitos.

Devuelve: una lista [nombre_c, edad, dni, mail]. Se devuelve como lista (no como cuatro valores sueltos) para poder pasarla entera a otras funciones más adelante.

Punto para la defensa: el while de validación no es un simple “if”; sigue reintentando hasta que el dato es válido, así el programa nunca avanza con datos corruptos.

seleccionar_estudio(especialidades, cardio, trauma, derma, pediatria) MAURI

Qué hace: muestra las 4 especialidades disponibles, el usuario elige una, y dentro de esa especialidad muestra las prácticas/estudios posibles para elegir una.

Técnicas usadas: - Lista por comprensión para armar las opciones numeradas: [f"{i+1}. {especialidades[i]}" for i in range(len(especialidades))] — arma “1. Cardiología”, “2. Traumatología”, etc. en una sola línea en vez de un for con append. - match-case sobre el número elegido (1-4) para decidir cuál lista de estudios usar (cardio, trauma, pediatria o derma). - El mismo patrón de comprensión de listas se repite para armar el menú de estudios dentro de la especialidad elegida.

Devuelve: una tupla implícita especialidad, estudio (dos strings) con el nombre de la especialidad y el nombre del estudio elegidos — no los números de opción, sino el texto ya resuelto.

Punto para la defensa: las especialidades y las listas de estudios de cada una (cardio, trauma, derma, pediatria) se arman en el programa principal y se pasan como parámetros — la función no tiene datos “hardcodeados” adentro, así que si mañana se agrega una especialidad nueva, no hay que tocar esta función.

mostrar_disponibilidad(matriz_turnos) JUANI

Qué hace: imprime la grilla completa de turnos: filas = días de la semana, columnas = franjas horarias, y en cada celda “Libre” u “Ocupado” según el estado de matriz_turnos.

Cómo arma la tabla: usa f"{valor:<N}" (alineación a la izquierda con ancho fijo) para que todas las columnas queden alineadas como una tabla real, tanto en el encabezado como en cada fila de días.

Punto para la defensa: es la función que muestra visualmente la matriz. No modifica nada, solo lee matriz_turnos y la imprime — separar “mostrar” de “modificar” (que lo hace elegir_turno) es una decisión de diseño para que cada función tenga una sola responsabilidad.

elegir_turno(matriz_turnos) JUANI

Qué hace: es el corazón del sistema de reservas. Muestra la grilla (llamando a mostrar_disponibilidad), hace elegir un día, calcula qué horarios están libres ese día, hace elegir un horario entre los libres, y marca esa celda de la matriz como “Ocupado”.

Cómo funciona paso a paso: 1. Pide un día (1 a 5, validado con while). 2. Con dia_idx = dia_opc - 1 convierte la opción elegida (1-based, pensada para el usuario) en índice de lista (0-based). 3. Arma horas_libres con una lista por comprensión que filtra FRANJAS_HORARIAS según cuáles están en “Libre” en la fila de ese día: matriz_turnos[dia_idx][h] == "Libre". 4. Si no hay horas libres ese día, avisa y vuelve a pedir otro día (continue, sin salir del while externo). 5. Si hay horas libres, las muestra y pide que el usuario tipee un horario (validado contra horas_libres, no puede elegir uno ocupado). 6. Con FRANJAS_HORARIAS.index(hora_elegida) recupera el índice de esa franja y hace matriz_turnos[dia_idx][franja_idx] = "Ocupado" — acá es donde la matriz efectivamente se actualiza.

Devuelve: DIAS_SEMANA[dia_idx], FRANJAS_HORARIAS[franja_idx] — el nombre del día y el horario elegidos.

Punto para la defensa: esta función es la prueba de que la matriz no es decorativa — es un sistema de reservas real: si dos pacientes distintos en la misma sesión intentan el mismo día/horario, el segundo ya no lo va a ver en `horas_libres` porque la matriz quedó modificada por el primero (`matriz_turnos` es la misma lista compartida en memoria, se pasa por referencia).

obra_social() MARTI

Qué hace: pregunta si el paciente tiene obra social/prepaga y calcula el monto a pagar.

Lógica: - Si responde “no”: `monto = precio_particular (15000)`, `cobertura = “Particular (Sin Obra Social)”`. - Si responde “si”: pide el nombre de la entidad (guardado en mayúsculas con `.upper()`), `monto = plus_coseguro (3500, un coseguro reducido en vez del precio completo)`. - Usa `match-case` sobre la respuesta “si”/“no” en vez de `if/elif`.

Devuelve: `nombre_os, monto_final` (string y número).

Punto para la defensa: los precios (`precio_particular`, `plus_coseguro`) son variables locales de la función, no constantes globales — si el profesor pregunta por qué, la respuesta es que solo se usan acá adentro, no hace falta exponerlas al resto del programa.

formatear_monto(monto) MAURI

Qué hace: función chica y pura (sin `input/print`) que recibe un número y devuelve un string con formato moneda: “\$15.000” en vez de “15000”.

Cómo: `f"${monto:,.0f}"` primero formatea con coma como separador de miles (estándar de Python/EEUU), y `.replace(",",".")` la cambia por punto (convención argentina).

Punto para la defensa: es un buen ejemplo de función auxiliar reutilizable — la usan tanto `generar_comprobante` como `mostrar_resumen_turnos`, así que si mañana cambia el formato de moneda, se edita en un solo lugar.

generar_comprobante(datos, especialidad, estudio, dia, hora, obra_social_nombre, monto)
MARTI

Qué hace: arma el comprobante final de la reserva con formato de ticket, lo imprime y lo devuelve como string.

Cómo arma el texto: - Desempaqueta `datos` (la lista que devolvió `datos_paciente`) en `nombre_c`, `edad`, `dni`, `mail`. - Construye una lista `lineas` con cada renglón del comprobante (separadores “=” * 45, título, cada dato). - Usa alineación con `f"{'Paciente:':<16}{nombre_c}"` para que las etiquetas queden alineadas en columna, como en un ticket real. - Llama a `formatear_monto(monto)` para el total. - Al final, `"\n".join(lineas)` arma un único string con saltos de línea — técnica de cadenas: separar la construcción del texto (lista de líneas) de su formato final (un string unido).

Devuelve: el comprobante completo como string (por si se quisiera reutilizar, aunque en este programa solo se imprime).

Punto para la defensa: es la función donde más se nota el trabajo con cadenas (f-strings, alineación, formateo) — vale la pena mostrarla en pantalla durante la defensa.

mostrar_resumen_turnos(lista_turnos) LULI

Qué hace: al final de la sesión, imprime una tabla resumen con todos los turnos reservados (uno por paciente atendido).

Cómo: - Si lista_turnos está vacía, avisa que no se registró ningún turno. - Si no, recorre cada turno (que es una lista de 7 elementos: nombre, dni, especialidad, estudio, día, hora, monto), lo desempaqueta y arma una fila con columnas alineadas (mismo patrón `f"{valor:<N}"` que en `mostrar_disponibilidad`).

Punto para la defensa: lista_turnos es una lista de listas — no es la matriz de turnos (que es la grilla día×hora), es un historial secuencial de reservas. Buen punto para diferenciar ante el profesor “matriz de estado” vs. “lista de registros”, que son dos usos distintos de listas anidadas.

4. Programa principal — flujo completo JUANI

1. Saludo inicial
2. Se definen las especialidades y sus listas de estudios (cardio, trauma, derma, pediatra)
3. Se crea matriz_turnos: todas las celdas en "Libre" (comprensión de listas anidada)
4. Se crea lista_turnos vacía
5. Loop principal (while continuar):
 - a. Muestra menú: 1) Atender nuevo paciente 2) Finalizar y ver resumen
 - b. match-case sobre la opción elegida:
 - Opción 1:
 - datos_paciente() → datos del paciente
 - seleccionar_estudio(...) → especialidad + estudio
 - elegir_turno(matriz_turnos) → día + hora (y marca la matriz como ocupada)
 - obra_social() → cobertura + monto
 - generar_comprobante(...) → imprime el ticket
 - se agrega el turno a lista_turnos
 - Opción 2:
 - continuar = False → corta el loop
6. Al salir del loop: mostrar_resumen_turnos(lista_turnos)

Por qué está armado así: cada iteración del loop repite el flujo completo de atención de un paciente sin perder el estado entre pacientes — la matriz de turnos y la lista de turnos reservados persisten de una vuelta a otra del while, porque se crean una sola vez antes del loop y las funciones las reciben por parámetro (no se recrean adentro).